ZARC DRT Derivation and Peak-Parameter Mapping

Souradeep Mitra

2026-07-21

1 Purpose

This note collects the mathematical discussion used in the MATLAB single-file DRT workflow:

1. the exact DRT of a ZARC element,
2. the resistance, time constant, and capacitance-style quantities extracted from a refined DRT peak,
3. the full-width-at-half-maximum (FWHM) relation that can be used to estimate the ZARC exponent n from peak width.

The implementation context is the single-file workflow `Impedance_analysis_singlefile.m`, which follows the same broad DRT workflow described in the recent parameter-selection literature and Bayesian DRT studies [1, 2, 3, 4].

2 Code-Level Workflow Summary

At a high level, the current analysis workflow executes the following steps for a raw multi-temperature impedance data file:

1. load the multi-temperature workbook and split it into three-column blocks,
2. convert magnitude-phase data into complex impedance,
3. remove invalid frequencies and the high-frequency inductive tail,
4. build the DRT projection matrices and select the regularization parameter by generalized cross-validation,
5. recover the DRT vector and reconstruct the fitted impedance,
6. refine the DRT peak structure using an RBF interpolation plus Gaussian basis fit,
7. convert refined peaks into exported R, C, and heuristic n values,
8. reconstruct an explicit series R-C-n impedance model from fitted peaks and compare it with the measured and DRT-fit Nyquist curves,
9. run Kramers-Kronig diagnostics on both measured impedance and DRT-reconstructed impedance,
10. run noise-injection sensitivity checks (multiple noise levels and random trials) around the selected fit,

11. accumulate consolidated arrays for raw/DRT/peak/RCN data,

12. after all temperatures are processed, generate global EPS figures and write consolidated Excel sheets (`RAW_ALL`, `DRT_ALL`, `PEAK_ALL`, `RCN_ALL`, plus summary matrices).

The code therefore has two complementary layers: a mathematical inversion layer and a reporting/visualization layer. The second layer is not cosmetic; it makes the method auditable temperature-by-temperature and preserves diagnostics needed to interpret regularization and peak-parameter stability [5, 1, 2].

3 Software and Reproducibility Note

The implementation is a single MATLAB script, `Impedance_analysis_singlefile.m`, written to run in a MATLAB 2011-compatible environment without requiring external toolboxes beyond the standard plotting and optimization capabilities used by the script. The input data are organized as temperature-wise three-column blocks $[f, |Z|, \theta]$ in a workbook such as `S2022Sap.xlsx`. The script converts each block into complex impedance, removes invalid and inductive-tail points, solves the regularized DRT inverse problem, refines peaks, and writes all intermediate and final products to Excel files with sample-tagged names.

This explicit workflow description matters because DRT software is sensitive not only to the mathematics of regularization, but also to implementation details such as frequency sorting, inductive-tail filtering, non-negativity enforcement, and output formatting. Recent DRT papers increasingly emphasize that parameter selection and implementation choices strongly influence the recovered relaxation spectrum, so the code path and exported diagnostics should be documented together with the equations [1, 2, 3, 4].

In practical terms, this note documents the mapping from the top-level script to the helper functions that implement preprocessing, DRT inversion, forward reconstruction, KK checking, and peak refinement. That makes the mathematics reproducible for another MATLAB user and also makes the output files traceable back to the source steps that created them.

4 Execution Flowchart

5 How the MATLAB Functions Map to the Mathematics

The current implementation is mostly an inline top-level loop plus a compact local-helper block. Matrix assembly, GCV scoring, and KK/noise summaries are computed directly inside the per-temperature loop; peak refinement and export-writing use local helpers.

Input and Preprocessing Helpers

- Workbook parsing uses inline `readmatrix`/`readcell` logic, with fallback handling for robust import.

- Frequency/magnitude/phase conversion to complex impedance is performed inline per temperature block.

- Inductive-tail removal is performed inline by trimming the contiguous high-frequency region where $\Im(Z) > 0$.

DRT Inversion and Forward Model Helpers

- A_{re} and A_{im} are assembled inline in the main loop from the measured frequency grid.
- Imaginary-branch and real-branch nonnegative ridge solves for GCV scoring are computed inline with `lsqnonneg`.
- DRT forward reconstruction is computed inline as $Z_{\mathrm{cal}} = R_\infty + A_{\mathrm{re}}\gamma + iA_{\mathrm{im}}\gamma$.
- `calculate_eis_from_rcn_peaks_2023`: reconstructs impedance from fitted series R-C-n peak branches.

Peak Refinement Helpers

- `create_rbf_model_2023` and `rbf_eval_2023`: interpolate the raw DRT onto a dense log-time grid.
- `find_peaks_simple_2023`: base local-maxima detection with height and spacing constraints.
- `detect_hidden_peaks_second_derivative_2023`: curvature-based shoulder/hidden-peak detection.
- `fit_rbf_gaussian_peaks_2023`, `solve_gaussian_rbf_sum_fit_2023`, and `evaluate_gaussian_rbf_sum_2` Gaussian-sum fitting and parameter extraction.
- `gaussian_fwhm_from_sigma_2023`: converts fitted σ to log-time FWHM.

Validation and Reporting Helpers

- KK candidate sweep and residual metrics are computed inline in the main loop.
- Noise sensitivity (multi-level, multi-trial residual statistics) is computed inline in the main loop.
- `prepare_excel_output_path_2023`: handles locked-output fallback by switching to a timestamped workbook name.
- `write_excel_block_2023`: writes consolidated sheets via non-COM Excel I/O.

6 DRT Representation

We use the DRT representation

$$Z(\omega) = R_\infty + \int_0^\infty \frac{\Gamma(\tau)}{1 + i\omega\tau}\, d\ln\tau. \tag{1}$$

This is the standard continuous relaxation-time representation used throughout the DRT literature for impedance spectra, dielectric spectra, and related ill-posed inverse problems [6, 7, 8, 9, 10].

Equivalently, if we define

$$y = \ln\left(\frac{\tau}{\tau_0}\right), \qquad \tau = \tau_0 e^y, \tag{2}$$

then

$$Z(\omega) = R_\infty + \int_{-\infty}^\infty \frac{\gamma(y)}{1 + i\omega\tau_0 e^y}\, dy, \tag{3}$$

where

$$\Gamma(\tau) = \gamma\left(\ln\frac{\tau}{\tau_0}\right). \tag{4}$$

7 Discrete Forward Model Used in the MATLAB Code

The MATLAB workflow discretizes the DRT integral on the measured frequency grid. If the experimental frequencies are

$$f_1, f_2, \ldots, f_N, \qquad \omega_p = 2\pi f_p, \tag{5}$$

and the corresponding relaxation times are approximated by

$$\tau_q \approx \frac{1}{f_q}, \tag{6}$$

then the real and imaginary parts of the impedance are approximated by matrix-vector products

$$\Re(Z) \approx R_\infty \mathbf{1} + A_{\mathrm{re}}\boldsymbol{\gamma}, \tag{7}$$

$$\Im(Z) \approx A_{\mathrm{im}}\boldsymbol{\gamma}. \tag{8}$$

Here $\boldsymbol{\gamma} \in \mathbb{R}^N$ is the discretized DRT vector and the matrices A_{re} and A_{im} are assembled in the MATLAB functions `calc_A_re_matlab2011` and `calc_A_im_matlab2011`. Their entries are

$$(A_{\mathrm{re}})_{pq} = -\frac{1}{2}\frac{\Delta \ln \tau_q}{1 + (\omega_p \tau_q)^2}, \tag{9}$$

$$(A_{\mathrm{im}})_{pq} = \frac{1}{2}\frac{\omega_p \tau_q}{1 + (\omega_p \tau_q)^2}\,\Delta \ln \tau_q, \tag{10}$$

where the code approximates the local log-width by neighboring values,

$$\Delta \ln \tau_q \approx \begin{cases} \ln(\tau_{q+1}/\tau_q), & q = 1, \\ \ln(\tau_q/\tau_{q-1}), & q = N, \\ \ln(\tau_{q+1}/\tau_{q-1}), & 1 < q < N. \end{cases} \tag{11}$$

This converts the continuous DRT inversion into a finite-dimensional constrained inverse problem, which is the same regularized discretization strategy used in practical DRT solvers and parameter-selection studies [6, 5, 1].

8 Tikhonov Regularization and Non-Negative DRT Recovery

The DRT inversion is ill-posed: small perturbations in the impedance data can lead to large oscillations in the recovered DRT. The MATLAB code therefore uses first-level Tikhonov regularization together with a non-negativity constraint.

The full inversion solved by `TR_DRT_matlab2011` is

$$\min_{\boldsymbol{\gamma} \geq 0,\, R_\infty} \left\| \begin{bmatrix} A_{\mathrm{re}} & \mathbf{1} \\ A_{\mathrm{im}} & \mathbf{0} \\ \sqrt{\lambda/2}\, I & \mathbf{0} \end{bmatrix} \begin{bmatrix} \boldsymbol{\gamma} \\ R_\infty \end{bmatrix} - \begin{bmatrix} \Re(\boldsymbol{Z}_{\mathrm{exp}}) \\ \Im(\boldsymbol{Z}_{\mathrm{exp}}) \\ \mathbf{0} \end{bmatrix} \right\|_2^2 . \tag{12}$$

Equivalently, this is the constrained minimization of

$$\|\Re(\boldsymbol{Z}_{\mathrm{exp}}) - (R_\infty \mathbf{1} + A_{\mathrm{re}}\boldsymbol{\gamma})\|_2^2 + \|\Im(\boldsymbol{Z}_{\mathrm{exp}}) - A_{\mathrm{im}}\boldsymbol{\gamma}\|_2^2 + \frac{\lambda}{2}\|\boldsymbol{\gamma}\|_2^2, \tag{13}$$

subject to $\boldsymbol{\gamma} \geq 0$.

The role of each term is:

- the first term fits the real part of the measured impedance,

- the second term fits the imaginary part,
- the third term penalizes large or oscillatory DRT amplitudes,
- the non-negativity constraint enforces a physically interpretable relaxation spectrum.

Larger λ produces smoother spectra with larger residuals, while smaller λ produces sharper spectra that risk overfitting measurement noise. This bias-variance tradeoff is the basic reason regularization-parameter selection is a central part of modern DRT analysis [11, 12, 5, 1, 2].

9 Residuals Used Throughout the Workflow

Once a candidate DRT $\boldsymbol{\gamma}$ and resistance R_∞ are found, the code reconstructs the fitted impedance

$$\boldsymbol{Z}_{\mathrm{cal}} = R_\infty \mathbf{1} + A_{\mathrm{re}}\boldsymbol{\gamma} + iA_{\mathrm{im}}\boldsymbol{\gamma}. \tag{14}$$

The complex residual vector is

$$\boldsymbol{r} = \boldsymbol{Z}_{\mathrm{exp}} - \boldsymbol{Z}_{\mathrm{cal}}. \tag{15}$$

The code reports summary residual metrics such as

$$\text{mean absolute residual} = \frac{1}{N}\sum_{p=1}^{N} |r_p|, \tag{16}$$

$$\text{max absolute residual} = \max_p |r_p|. \tag{17}$$

10 Component-Wise Inversion and Cross-Validation Logic

The MATLAB code does not choose λ from the full Tikhonov solve directly. Instead, it evaluates each candidate λ by solving two component-wise problems.

For the imaginary-only branch, the code solves

$$\min_{\boldsymbol{\gamma}\geq 0} \left\| \begin{bmatrix} A_{\mathrm{im}} \\ \sqrt{\lambda/2}\, I \end{bmatrix} \boldsymbol{\gamma} - \begin{bmatrix} \Im(\boldsymbol{Z}_{\mathrm{exp}}) \\ \mathbf{0} \end{bmatrix} \right\|_2^2. \tag{18}$$

Then R_∞ is estimated afterwards by averaging the real-part mismatch:

$$R_\infty = \frac{1}{N}\sum_{p=1}^{N} \left(\Re(Z_{\mathrm{exp},p}) - (A_{\mathrm{re}}\boldsymbol{\gamma})_p\right). \tag{19}$$

For the real-only branch, the code solves

$$\min_{\boldsymbol{\gamma}\geq 0} \left\| \begin{bmatrix} A_{\mathrm{re}} \\ \sqrt{\lambda/2}\, I \end{bmatrix} \boldsymbol{\gamma} - \begin{bmatrix} \Re(\boldsymbol{Z}_{\mathrm{exp}}) \\ \mathbf{0} \end{bmatrix} \right\|_2^2. \tag{20}$$

This gives two channel-specific DRT estimates. The workflow then uses both real and imaginary residuals to assess whether the selected λ generalizes across components rather than merely fitting one channel in isolation.

11 Generalized Cross-Validation Criterion

For each candidate λ, the code forms the stacked residual norm

$$\mathrm{RSS}(\lambda) = \left\| \begin{bmatrix} \Re(\boldsymbol{Z}_{\mathrm{exp}}) - (R_\infty \mathbf{1} + A_{\mathrm{re}}\boldsymbol{\gamma}) \\ \Im(\boldsymbol{Z}_{\mathrm{exp}}) - A_{\mathrm{im}}\boldsymbol{\gamma} \end{bmatrix} \right\|_2^2. \tag{21}$$

Let N be the number of frequencies and $M = 2N$ the total number of real-valued data equations. The code computes a normalized GCV score of the form

$$\mathrm{nGCV}(\lambda) = \frac{\mathrm{RSS}(\lambda)/M}{\left(1 - \mathrm{tr}(H_\lambda)/N\right)^2}, \tag{22}$$

where H_λ is the effective hat matrix of the corresponding ridge problem.

For the imaginary-only branch, the code uses

$$H_\lambda^{(\mathrm{im})} = A_{\mathrm{im}} \left(A_{\mathrm{im}}^T A_{\mathrm{im}} + \frac{\lambda}{2} I \right)^{-1} A_{\mathrm{im}}^T, \tag{23}$$

and analogously for the real-only branch,

$$H_\lambda^{(\mathrm{re})} = A_{\mathrm{re}} \left(A_{\mathrm{re}}^T A_{\mathrm{re}} + \frac{\lambda}{2} I \right)^{-1} A_{\mathrm{re}}^T. \tag{24}$$

The traces $\mathrm{tr}(H_\lambda)$ are computed in the MATLAB helpers `hat_trace_impart_matlab2011` and `hat_trace_repart_matlab2011`. They act as effective degrees of freedom: larger trace means a more flexible fit.

Conceptually, GCV balances two effects:

- small residual error favors small λ,
- low effective complexity favors larger λ.

The denominator penalizes models whose effective degrees of freedom approach the data count.

This is the main reason the workflow prefers GCV over the L-curve criterion. GCV is derived as an estimate of the leave-one-out prediction error, so it directly asks which λ best predicts data that were not used in the fit. That is the relevant objective for DRT inversion, where the goal is a stable impedance representation that generalizes across frequency points and across datasets [11, 5, 1].

The L-curve uses the corner of the parametric curve $(\|A\gamma - b\|_2, \|\gamma\|_2)$. In practice, the location of that corner can shift with the discretization of the relaxation-time grid, the amount of noise, and the numerical rule used to detect curvature. That makes the L-curve more subjective and less reproducible in large automated sweeps.

By contrast, GCV is a scalar score with a clear statistical interpretation: the residual term rewards fidelity, while the hat-matrix trace estimates the effective degrees of freedom and penalizes overfitting. Bayesian DRT methods are attractive when one wants a full posterior distribution, uncertainty quantification, or explicit prior information about smoothness or sparsity [3, 2]. However, they require a prior or hyperprior specification and are usually more expensive to run and tune across many datasets. For this workflow, the objective is not posterior inference but a stable regularization level that can be chosen automatically and consistently across temperatures and samples. GCV does that with one deterministic score and no prior tuning, so it is the better default for the present impedance-inversion pipeline [11, 12, 3, 5, 4, 1, 2].

12 GCV Slope Criterion and Lambda Selection Policy

The code computes the discrete log-slope of the GCV curve,

$$\frac{d\ln G}{d\ln \lambda} \approx \frac{\ln G_{i+1} - \ln G_i}{\ln \lambda_{i+1} - \ln \lambda_i}, \tag{25}$$

where $G_i = \mathrm{nGCV}(\lambda_i)$.

This slope is used to detect a flat region of the GCV curve. In the code, a segment is considered flat when

$$\left|\frac{d\ln G}{d\ln \lambda}\right| \leq 0.002. \tag{26}$$

If such a flat region exists, the workflow selects the largest λ in that flat zone. This policy prefers the smoothest DRT that still lies on the GCV plateau. If no flat region is detected, the code falls back to the minimum-GCV solution:

$$\lambda_{\mathrm{best}} = \arg\min_{\lambda} \mathrm{nGCV}(\lambda). \tag{27}$$

Thus the MATLAB selection policy is a hybrid of plateau detection and standard GCV minimization, chosen to avoid over-interpreting small numerical differences between nearby admissible regularization levels [5, 1, 2].

13 Robustness Checks in the Current Script

After selecting λ by the GCV policy, the current script performs a direct noise-injection sensitivity check around the selected fit. For each noise level (e.g., 0.5% to 10%), several random perturbations are added to the complex impedance data, the inversion is re-solved at the selected λ, and residual percentages are summarized.

This is different from a full bootstrap uncertainty pipeline: it is a fast operational robustness check designed for large temperature sweeps. The output answers a practical question: if experimental noise increases, how quickly does fit quality degrade at the chosen regularization level? This keeps runtime low while still reporting stability diagnostics that can be compared across temperatures and samples [13, 6, 1, 2].

14 How the EPS Images Are Formed

The script generates each published image from arrays accumulated during the temperature loop, then saves vector graphics at the end with `print(...,'-depsc')`. The active figures are:

1. `_peak_XXXK.eps`: per-temperature DRT peak plot (RBF curve, Gaussian fit, base/hidden markers).
2. `_rgb_peak_map.eps`: RGB map where channels encode ranked peak components across temperature.
3. `_ngcv_imag_colormap.eps`: temperature-vs-λ nGCV matrix for imaginary-branch inversion.
4. `_ngcv_real_colormap.eps`: temperature-vs-λ nGCV matrix for real-branch inversion.
5. `_combined_residual_lambda_temp.eps`: combined residual map with selected-λ overlay.
6. `_noise_mean_residual_colormap.eps`: temperature-vs-noise-level mean residual map.

7. `_kk_raw_residual_vs_temp.eps`: KK residual trends over temperature.

The RGB map is formed in three steps:

1. For each temperature, identify up to three dominant fitted peaks.
2. Convert each peak component to a dense log-time profile over a common axis.
3. Normalize and assign channel intensities (R/G/B) by ranked peak contribution, then stack all temperatures as image rows.

15 Kramers-Kronig Consistency Test Used in the Code

The KK test in the MATLAB file fits the impedance with a finite set of Debye-like branches plus series resistance and inductance. For a candidate basis size n_b and regularization weight α, the model is

$$Z_{\mathrm{KK}}(\omega) = R_\infty + \sum_{j=1}^{n_b} \frac{R_j}{1 + i\omega\tau_j} + i\omega L. \tag{28}$$

In real and imaginary parts this is represented by matrices B_{re} and B_{im} whose entries are

$$(B_{\mathrm{re}})_{pj} = \frac{1}{1 + (\omega_p\tau_j)^2}, \tag{29}$$

$$(B_{\mathrm{im}})_{pj} = -\frac{\omega_p\tau_j}{1 + (\omega_p\tau_j)^2}. \tag{30}$$

The constrained least-squares problem solved by the code is

$$\min_{\boldsymbol{R}\geq 0,\, R_\infty,\, L} \left\| \begin{bmatrix} B_{\mathrm{re}} & \mathbf{1} & \mathbf{0} \\ B_{\mathrm{im}} & \mathbf{0} & \boldsymbol{\omega} \\ \sqrt{\alpha} I & \mathbf{0} & \mathbf{0} \end{bmatrix} \begin{bmatrix} \boldsymbol{R} \\ R_\infty \\ L \end{bmatrix} - \begin{bmatrix} \Re(\boldsymbol{Z}_{\mathrm{exp}}) \\ \Im(\boldsymbol{Z}_{\mathrm{exp}}) \\ \mathbf{0} \end{bmatrix} \right\|_2^2. \tag{31}$$

The code tests several candidate basis sizes and regularization values, retains the best solution by total residual percentage, and reports

$$\text{real residual } \% = 100 \frac{\|\Re(\boldsymbol{r})\|_2}{\|\Re(\boldsymbol{Z}_{\mathrm{exp}})\|_2}, \tag{32}$$

$$\text{imag residual } \% = 100 \frac{\|\Im(\boldsymbol{r})\|_2}{\|\Im(\boldsymbol{Z}_{\mathrm{exp}})\|_2}, \tag{33}$$

$$\text{total residual } \% = 100 \frac{\|[\Re(\boldsymbol{r}); \Im(\boldsymbol{r})]\|_2}{\|[\Re(\boldsymbol{Z}_{\mathrm{exp}}); \Im(\boldsymbol{Z}_{\mathrm{exp}})]\|_2}. \tag{34}$$

The same KK procedure is applied both to the measured impedance and to the DRT-reconstructed impedance as a consistency check on the inversion. That is a useful implementation-level validation because it checks whether the regularized DRT solution remains compatible with a causal passive response, not merely whether it gives a small fitting residual [8, 7].

16 ZARC Element

A ZARC element may be written as a resistor R in parallel with a constant-phase element (CPE):

$$Y_{\mathrm{CPE}} = Q(i\omega)^n, \qquad 0 < n \leq 1. \tag{35}$$

The total admittance is

$$Y(\omega) = \frac{1}{R} + Q(i\omega)^n, \tag{36}$$

so the impedance is

$$Z(\omega) = \frac{1}{Y(\omega)} \tag{37}$$

$$= \frac{R}{1 + RQ(i\omega)^n}. \tag{38}$$

Defining the characteristic time

$$\tau_0 = (RQ)^{1/n}, \tag{39}$$

the ZARC becomes

$$Z_{\mathrm{ZARC}}(\omega) = \frac{R}{1 + (i\omega\tau_0)^n}. \tag{40}$$

In the MATLAB reconstruction used in the workflow, each refined arc is written as

$$Z_k(\omega) = \frac{R_k}{1 + (i\omega R_k C_k)^{n_k}}, \tag{41}$$

which identifies an effective RC-style time constant

$$\tau_k = R_k C_k. \tag{42}$$

This ZARC parameterization is widely used in impedance modeling and in DRT-based interpretations of dielectric and electrochemical spectra [9, 7, 8].

17 Exact DRT of a ZARC

We seek a DRT density that reproduces

$$Z(\omega) = \frac{R}{1 + (i\omega\tau_0)^n}. \tag{43}$$

Define the analytic continuation

$$F(s) = \frac{R}{1 + (s\tau_0)^n}, \qquad \Re(s) > 0. \tag{44}$$

The DRT density follows from the Stieltjes inversion formula

$$\Gamma(\tau) = -\frac{1}{\pi}\,\Im\left[F\left(-\frac{1}{\tau} + i0\right)\right]. \tag{45}$$

Set

$$u = \left(\frac{\tau_0}{\tau}\right)^n. \tag{46}$$

On the negative real axis, using the principal branch,

$$\left(-\frac{\tau_0}{\tau} + i0\right)^n = u e^{i\pi n}. \tag{47}$$

Therefore,

$$F\left(-\frac{1}{\tau} + i0\right) = \frac{R}{1 + u e^{i\pi n}} = \frac{R}{1 + u\cos(\pi n) + iu\sin(\pi n)}. \tag{48}$$

Taking the imaginary part gives

$$\Im(F) = -\frac{Ru\sin(\pi n)}{1 + 2u\cos(\pi n) + u^2}. \tag{49}$$

Hence the exact DRT is

$$\Gamma(\tau) = \frac{R}{\pi}\frac{(\tau_0/\tau)^n \sin(\pi n)}{1 + 2\,(\tau_0/\tau)^n \cos(\pi n) + (\tau_0/\tau)^{2n}}. \tag{50}$$

Using $y = \ln(\tau/\tau_0)$, so that $u = e^{-ny}$, this becomes the symmetric log-time form

$$\gamma(y) = \frac{R\sin(\pi n)}{2\pi\left[\cosh(ny) + \cos(\pi n)\right]}. \tag{51}$$

This is the exact DRT of a ZARC element, and the closed-form log-time density is a standard reference result for connecting ZARC fits to continuous relaxation spectra [9, 10].

18 Derivation: ZARC as an Infinite Series of ZAP Elements

In this note, define one elementary ZAP element as a single Debye-type relaxation contribution in series,

$$Z_{\mathrm{ZAP}}(\omega;\tau,\Delta R) = \frac{\Delta R}{1 + i\omega\tau}, \tag{52}$$

where $\Delta R > 0$ is the resistance weight carried by that relaxation time.

The DRT representation of a passive spectrum is

$$Z(\omega) = R_\infty + \int_0^\infty \frac{\Gamma(\tau)}{1 + i\omega\tau}\, d\ln\tau. \tag{53}$$

Now partition the $\ln\tau$ axis into bins with centers τ_k and widths $\Delta\ln\tau_k$. Then

$$Z(\omega) - R_\infty \approx \sum_{k=1}^{K} \frac{\Gamma(\tau_k)\,\Delta\ln\tau_k}{1 + i\omega\tau_k} = \sum_{k=1}^{K} \frac{\Delta R_k}{1 + i\omega\tau_k}, \tag{54}$$

with

$$\Delta R_k := \Gamma(\tau_k)\,\Delta\ln\tau_k. \tag{55}$$

Each summand is exactly one ZAP element in series. Taking the refinement limit $\max_k \Delta\ln\tau_k \to 0$ gives

$$Z(\omega) - R_\infty = \lim_{K\to\infty} \sum_{k=1}^{K} \frac{\Delta R_k}{1 + i\omega\tau_k}, \tag{56}$$

which proves that any DRT-representable impedance is the limit of an infinite series of ZAP elements.

For a ZARC specifically,

$$Z_{\mathrm{ZARC}}(\omega) = \frac{R}{1 + (i\omega\tau_0)^n}, \tag{57}$$

and its associated DRT density is

$$\Gamma(\tau) = \frac{R}{\pi}\frac{(\tau_0/\tau)^n \sin(\pi n)}{1 + 2\,(\tau_0/\tau)^n \cos(\pi n) + (\tau_0/\tau)^{2n}}. \tag{58}$$

Substituting this density into the integral above gives an exact continuum superposition of Debye/ZAP terms with weight

$$dR(\tau) = \Gamma(\tau)\, d\ln\tau, \tag{59}$$

and total distributed resistance

$$\int_0^\infty \Gamma(\tau)\, d\ln\tau = R. \tag{60}$$

So, in the precise mathematical sense used by DRT, one ZARC element is equivalent to an infinite series (continuum sum) of ZAP elements over relaxation time.

19 Area Under the Peak and Resistance

In DRT, the resistance contribution of a relaxation process is the area under the peak with respect to $d\ln\tau$:

$$R = \int_{-\infty}^{\infty} \gamma(y)\, dy. \tag{61}$$

The reason is that resistance is defined from the *DC limit* of impedance. Starting from

$$Z(\omega) - R_\infty = \int_0^\infty \frac{\Gamma(\tau)}{1 + i\omega\tau}\, d\ln\tau, \tag{62}$$

set $\omega \to 0$ to obtain

$$Z(0) - R_\infty = \int_0^\infty \Gamma(\tau)\, d\ln\tau. \tag{63}$$

So the total distributed resistance is exactly the unweighted area under the DRT.

By contrast,

$$\int_0^\infty \frac{\Gamma(\tau)}{1 + (\omega\tau)^2}\, d\ln\tau \tag{64}$$

is a frequency-dependent quantity: it is the real-part weighting kernel at a specific nonzero ω, not the total resistance parameter. Therefore it cannot replace $\int \Gamma\, d\ln\tau$ as the definition of resistance.

In the workflow, each refined peak is approximated locally by a Gaussian in log-time,

$$\gamma_{\text{fit}}(y) = A \exp\left[-\frac{(y-\mu)^2}{2\sigma^2}\right]. \tag{65}$$

The area under this Gaussian is

$$\int_{-\infty}^{\infty} A \exp\left[-\frac{(y-\mu)^2}{2\sigma^2}\right] dy = A\sigma\sqrt{2\pi}. \tag{66}$$

This gives the resistance estimate used in the code:

$$R_{\text{est}} = A\sigma\sqrt{2\pi}. \tag{67}$$

20 Peak Detection, RBF Refinement, and Gaussian Peak Model

The peak-analysis stage is not performed on the raw DRT vector directly. Instead, the code first interpolates the recovered DRT over a dense log-time grid using a Gaussian radial basis function model,

$$\gamma_{\text{RBF}}(x) = \sum_{j=1}^{N} w_j \exp\left[-(\varepsilon(x - x_j))^2\right], \tag{68}$$

where $x = \ln\tau$.

The base peaks are then detected by local-maxima rules with a minimum height threshold and a minimum separation in sample index. Hidden or shoulder peaks are identified by analyzing

the smoothed negative second derivative of the RBF curve. In other words, the code uses curvature maxima in

$$-\frac{d^2\gamma_{\mathrm{RBF}}}{dx^2} \tag{69}$$

to locate shoulders that may not appear as strong raw maxima.

After combining base peaks and hidden peaks, the code fits a global Gaussian sum with baseline,

$$\gamma_{\mathrm{fit}}(x) = \sum_{k=1}^{m} A_k \exp\left[-\frac{(x-\mu_k)^2}{2\sigma_k^2}\right] + b, \tag{70}$$

where the centers μ_k are inherited from the refined peak-detection step and the amplitudes, widths, and baseline are optimized numerically.

The objective minimized in the code is a penalized least-squares function,

$$\mathrm{SSE} = \sum_j (\gamma_{\mathrm{fit}}(x_j) - \gamma_{\mathrm{RBF}}(x_j))^2 + \text{width penalties} + \text{small-amplitude penalties}. \tag{71}$$

This keeps the fitted Gaussian components compact and suppresses numerically unstable peak widths.

21 Time Constant and Capacitance-Style Parameter

The fitted center gives the peak relaxation time:

$$\tau_{\mathrm{peak}} = e^{\mu}. \tag{72}$$

The workflow then forms an effective capacitance-style quantity from

$$\tau_{\mathrm{peak}} = R_{\mathrm{est}} C_{\mathrm{est}}, \tag{73}$$

so that

$$C_{\mathrm{est}} = \frac{\tau_{\mathrm{peak}}}{R_{\mathrm{est}}}. \tag{74}$$

For a true ZARC, the parallel element is a CPE with parameter Q, so C_{est} should be read as an effective RC-style summary rather than an exact ideal capacitance whenever $n < 1$. That interpretation is consistent with the broader ZARC/CPE literature in impedance spectroscopy [9, 7].

22 Heuristic Calculation of R, C, and n in the Present MATLAB Workflow

For each Gaussian component recovered from the refined peak fit, the code exports the following quantities:

$$R_k = A_k \sigma_k \sqrt{2\pi}, \tag{75}$$

$$\tau_k = e^{\mu_k}, \tag{76}$$

$$C_k = \frac{\tau_k}{R_k}, \tag{77}$$

$$n_k = \frac{1.144}{1+\sigma_k}. \tag{78}$$

The resistance expression is tied to the DC limit of the DRT model, not to a finite-frequency kernel. Starting from

$$Z(\omega) - R_\infty = \int \frac{\Gamma(\tau)}{1 + i\omega\tau}\, d\ln\tau, \tag{79}$$

take $\omega \to 0$:

$$Z(0) - R_\infty = \int \Gamma(\tau)\, d\ln\tau. \tag{80}$$

So the total distributed resistance is the unweighted area under the DRT. In contrast,

$$\int \frac{\Gamma(\tau)}{1 + (\omega\tau)^2}\, d\ln\tau \tag{81}$$

is the real-part contribution at a specific finite ω:

$$\Re Z(\omega) - R_\infty = \int \frac{\Gamma(\tau)}{1 + (\omega\tau)^2}\, d\ln\tau. \tag{82}$$

Therefore the code uses peak area in $d\ln\tau$ to estimate resistance, while the frequency-weighted integral is used to interpret real-impedance contribution versus frequency.

The first three relations follow naturally from the Gaussian approximation to the DRT peak: area gives resistance, the center gives relaxation time, and $\tau = RC$ supplies an effective capacitance. The final relation for n is heuristic in the present code. It enforces the trend that broader peaks imply lower n, but it is not derived from the Tikhonov inversion itself. In the published DRT literature, similar post-processing fits are often used as interpretation aids rather than as part of the inversion objective [4, 5, 1].

23 FWHM-Based n Formula and Check of the $1.144/n$ Expression

From the exact ZARC DRT profile, the width relation is

$$\mathrm{FWHM}_{\mathrm{Gauss}} = 2\sqrt{2\ln 2}\,\sigma, \qquad \mathrm{FWHM}_{\mathrm{ZARC}}(n) = \frac{2}{n}\operatorname{arcosh}\left(2 + \cos(\pi n)\right), \tag{83}$$

and the theory-based n is defined implicitly by

$$\mathrm{FWHM}_{\mathrm{ZARC}}(n) = \mathrm{FWHM}_{\mathrm{Gauss}}. \tag{84}$$

So the mathematically correct FWHM-based statement is an implicit equation in n, solved numerically; it is *not* a direct closed form like $n = 1.144/\sigma$ or $n = 1.144/n$. This is consistent with the more general observation in the recent regularization-selection literature that many parameter mappings are empirical summaries and should be checked against the exact model shape when a closed-form relationship is available [1, 2].

For the current MATLAB heuristic,

$$n = \frac{1.144}{1 + \sigma}, \tag{85}$$

the exact rearrangement is

$$\sigma = \frac{1.144}{n} - 1. \tag{86}$$

Therefore, if one writes only "$1.144/n$", that is incomplete for this heuristic mapping. The correct relation needs the minus one term.

As an additional approximation tied to the ZARC-width derivation near $n \approx 1$, one commonly gets a rational form

$$n \approx \frac{a}{a + \sigma}, \qquad a \approx \frac{\pi}{\sqrt{2}} \approx 2.221, \tag{87}$$

which has the same qualitative trend as the heuristic but with a theory-derived scale constant.

24 Exact FWHM of the ZARC DRT

The peak maximum occurs at $y = 0$:

$$\gamma(0) = \frac{R\sin(\pi n)}{2\pi[1+\cos(\pi n)]}. \tag{88}$$

Define the normalized peak shape

$$f(y) = \frac{\gamma(y)}{\gamma(0)} = \frac{1+\cos(\pi n)}{\cosh(ny)+\cos(\pi n)}. \tag{89}$$

The half-maximum condition is

$$f(y_{1/2}) = \frac{1}{2}. \tag{90}$$

Therefore,

$$\frac{1+\cos(\pi n)}{\cosh(ny_{1/2})+\cos(\pi n)} = \frac{1}{2}, \tag{91}$$

$$2[1+\cos(\pi n)] = \cosh(ny_{1/2})+\cos(\pi n), \tag{92}$$

$$\cosh(ny_{1/2}) = 2+\cos(\pi n). \tag{93}$$

Thus,

$$y_{1/2} = \frac{1}{n}\operatorname{arcosh}\left(2+\cos(\pi n)\right), \tag{94}$$

and the exact full width at half maximum in log-time is

$$\boxed{\mathrm{FWHM}_{\mathrm{ZARC}}(n) = \frac{2}{n}\operatorname{arcosh}\left(2+\cos(\pi n)\right)}. \tag{95}$$

25 FWHM of the Gaussian Peak Fit

For a Gaussian

$$g(y) = A\exp\left[-\frac{(y-\mu)^2}{2\sigma^2}\right], \tag{96}$$

the full width at half maximum is

$$\boxed{\mathrm{FWHM}_{\mathrm{Gauss}} = 2\sqrt{2\ln 2}\,\sigma}. \tag{97}$$

26 Theory-Based Estimate of n from the Fitted Peak Width

To map the refined Gaussian width to a ZARC exponent from theory, one may equate the Gaussian FWHM to the exact ZARC DRT FWHM:

$$2\sqrt{2\ln 2}\,\sigma = \frac{2}{n}\operatorname{arcosh}\left(2+\cos(\pi n)\right). \tag{98}$$

Equivalently,

$$\sqrt{2\ln 2}\,\sigma = \frac{1}{n}\operatorname{arcosh}\left(2+\cos(\pi n)\right). \tag{99}$$

This relation is implicit in n. There is no simple closed-form algebraic inverse, so a theory-based n can be computed numerically by solving

$$g(n) = \frac{2}{n}\operatorname{arcosh}\left(2+\cos(\pi n)\right) - 2\sqrt{2\ln 2}\,\sigma = 0 \tag{100}$$

for $0 < n \leq 1$.

27 Current Code Formula for n

The current single-file MATLAB workflow keeps the heuristic form

$$n \approx \frac{1.144}{1+\sigma} \tag{101}$$

for the exported peak parameter n. This form is not derived from the exact ZARC DRT above. It only imposes the qualitative trend that broader DRT peaks should correspond to smaller n.

The FWHM-based relation derived in this note is theory-linked because it comes directly from the exact DRT shape of the ZARC element, but it is presented here as a derivation and alternative mapping, not as the active formula in the present single-file code. The distinction between heuristic and derivation-based post-processing is important in DRT work because the chosen interpretation formula should not be conflated with the inversion itself [9, 4, 1].

28 Summary of Formulas

For each refined peak:

$$R_{\text{est}} = A\sigma\sqrt{2\pi}, \tag{102}$$

$$\tau_{\text{peak}} = e^{\mu}, \tag{103}$$

$$C_{\text{est}} = \frac{\tau_{\text{peak}}}{R_{\text{est}}}, \tag{104}$$

$$\text{FWHM}_{\text{Gauss}} = 2\sqrt{2\ln 2}\,\sigma, \tag{105}$$

$$\text{FWHM}_{\text{ZARC}}(n) = \frac{2}{n}\operatorname{arcosh}\left(2+\cos(\pi n)\right), \tag{106}$$

$$n_{\text{heuristic}} = \frac{1.144}{1+\sigma}. \tag{107}$$

If one wants a theory-based alternative, then n_{est} may instead be defined implicitly by

$$\text{FWHM}_{\text{ZARC}}(n_{\text{est}}) = \text{FWHM}_{\text{Gauss}}. \tag{108}$$

Thus the present code uses the heuristic $n_{\text{heuristic}}$, while the FWHM-based relation in this note gives the derivation-based alternative. The code keeps both the heuristic output and the exact derivation in the note so the heuristic can be checked against a model-based alternative when needed [1, 2].

29 Code Output Inventory

For each input workbook, the current script writes one consolidated workbook plus EPS figures:

- `_postprocess_data.xlsx`: consolidated sheets `RAW_ALL`, `DRT_ALL`, `PEAK_ALL`, `RCN_ALL`, `SUMMARY`, `NGCV_IMAG`, `NGCV_REAL`, `RESID_LAMBDA`, `NOISE_MEAN`, `KK_SUMMARY`, and optional `RGB_map_data`,
- `figures_eps/`: vector images for per-temperature peak plots and global diagnostic maps,
- terminal/console log text from the MATLAB batch run.

This consolidated output design reduces workbook fragmentation and keeps the full run trace in one artifact, while the EPS directory preserves publication-quality vector plots for reporting and comparison [5, 1, 2].

30 Detailed RBF Interpolation and Gaussian Basis Fitting Theory

30.1 Radial Basis Function Model

The first refinement step constructs a smooth continuous model of the DRT using Gaussian radial basis functions (RBF). Given discrete DRT points $\{x_j, y_j\}$ where $x_j = \ln(\tau_j)$ and $y_j = \gamma(\tau_j)$, the RBF model is

$$\gamma_{\mathrm{RBF}}(x) = \sum_{j=1}^{N} w_j \exp\left[-(\varepsilon \cdot d_j)^2\right], \tag{109}$$

where $d_j = |x - x_j|$ is the Euclidean distance and ε is a shape parameter that determines the width of each Gaussian basis. This smooth refinement is a post-processing step for interpretation and peak localization rather than part of the regularized inversion itself [4, 1].

The shape parameter is computed from the data spacing as

$$\varepsilon = \frac{1}{\Delta x_{\mathrm{med}}}, \tag{110}$$

where Δx_{med} is the median spacing between sorted training points x_j.

The weights w_j are obtained by solving the regularized least-squares system,

$$(\mathbf{\Phi} + \lambda_{\mathrm{RBF}}\mathbf{I})\,\mathbf{w} = \mathbf{y}, \tag{111}$$

where $\mathbf{\Phi}$ is the Gaussian kernel matrix with entries $\Phi_{ij} = \exp[-(\varepsilon d_{ij})^2]$, $\lambda_{\mathrm{RBF}} = 10^{-8}$ is a small Tikhonov regularization parameter for numerical stability, and $\mathbf{y}$ is the vector of DRT values.

30.2 Base Peak Detection

Base peaks are identified by simple local-maxima detection on the RBF-interpolated curve. A point i is marked as a local maximum if

$$\gamma_{\mathrm{RBF}}(x_i) \geq \gamma_{\mathrm{RBF}}(x_{i-1}) \quad \text{and} \quad \gamma_{\mathrm{RBF}}(x_i) > \gamma_{\mathrm{RBF}}(x_{i+1}), \tag{112}$$

with a minimum height threshold

$$\gamma_{\mathrm{RBF}}(x_i) \geq h_{\min} = 0.05 \cdot \max_j \gamma_{\mathrm{RBF}}(x_j). \tag{113}$$

Peaks are then sorted by height in descending order and culled to enforce minimum separation: if two peaks are within $\Delta x_{\mathrm{sep}} = 20$ sample indices, only the taller peak is retained.

30.3 Hidden Peak Detection via Second Derivative

Hidden or shoulder peaks are identified by analyzing the smoothed negative second derivative (curvature) of the RBF curve. The curvature signal is defined as

$$\kappa(x_i) = \max\left(0, -\frac{d^2\gamma_{\mathrm{RBF}}}{dx^2}\right), \tag{114}$$

capturing regions of concavity. This signal is smoothed using a Savitzky-Golay filter (frame length 7, polynomial order 2).

Peaks in the curvature signal are detected with a threshold

$$\kappa_{\min} = \max\left(0.3 \cdot \max_j \kappa(x_j), 3.2 \cdot \sigma_\kappa\right), \tag{115}$$

where σ_κ is the standard deviation of $\kappa(x)$.

Each curvature peak passes multi-stage filters: proximity to base peaks, minimum amplitude, local floor, prominence, and interior checks. The final set is capped at $\lfloor m_b/2 \rfloor$ to prevent over-segmentation.

30.4 Gaussian Sum Fitting

The code fits a parametric Gaussian sum plus baseline:

$$\gamma_{\text{fit}}(x) = \sum_{k=1}^{m} A_k \exp\left[-\frac{(x-\mu_k)^2}{2\sigma_k^2}\right] + b, \tag{116}$$

where centers μ_k are fixed at detected peak locations and amplitudes, widths, and baseline are optimized.

The penalized least-squares objective is

$$\mathcal{J}(\mathbf{A}, \boldsymbol{\sigma}, b) = \sum_j [\gamma_{\text{fit}}(x_j) - \gamma_{\text{RBF}}(x_j)]^2 + 10^3 \cdot P_\sigma(\boldsymbol{\sigma}) + 10^{-2} \cdot P_A(\mathbf{A}), \tag{117}$$

with width penalty enforcing $\sigma_k \in [0.04, 0.90]$ and amplitude penalty enforcing positivity. Optimization uses the simplex method, and given optimal widths, amplitudes are obtained by non-negative least squares.

30.5 Per-Peak Circuit Parameter Extraction

30.5.1 Resistance from Peak Area

The resistance is the integral of the DRT:

$$R_k = A_k \sigma_k \sqrt{2\pi}. \tag{118}$$

30.5.2 Characteristic Time and Capacitance

The characteristic relaxation time is the peak center:

$$\tau_k = \exp(\mu_k), \quad C_k = \frac{\tau_k}{R_k}. \tag{119}$$

30.5.3 CPE Exponent Estimation

The **empirically optimal** formula (tested on S2022 data, 2.99% lower Nyquist residuals) is the heuristic [4, 5, 1]:

$$n_k = \frac{1.144}{1 + \sigma_k}. \tag{120}$$

Alternative formulas for comparison:

FWHM-based implicit formula: Solving numerically from

$$\sqrt{2\ln 2} \cdot \sigma_k \cdot n_k = \operatorname{acosh}(2 + \cos(\pi n_k)). \tag{121}$$

Phase-angle method:

$$n_k = -\frac{4\phi_{\text{apex}}}{\pi}, \tag{122}$$

where $\phi_{\text{apex}} = -\pi n/4$ is the phase angle at peak impedance.

References

[1] A. Maradesa; B. Py; T. H. Wan; M. B. Effat; F. Ciucci. Selecting the Regularization Parameter in the Distribution of Relaxation Times. *J. Electrochem. Soc.* **2023**, *170* (3), 030502. https://doi.org/10.1149/1945-7111/acbca4.

[2] P. Carbone; A. De Angelis; A. Bertei; A. Maradesa; F. Ciucci. Optimal Regularization for the Distribution of Relaxation Times via Frequency-Band Selection. *J. Electrochem. Soc.* **2025**, *172* (2), 020533. https://doi.org/10.1149/1945-7111/adb5c6.

[3] F. Ciucci; C. Chen. Analysis of Electrochemical Impedance Spectroscopy Data Using the Distribution of Relaxation Times: A Bayesian and Hierarchical Bayesian Approach. *Electrochim. Acta* **2015**, *167*, 439–454. https://doi.org/10.1016/j.electacta.2015.03.123.

[4] M. A. Danzer; S. M. Schmidt; R. Zeis. Generalized Distribution of Relaxation Times Analysis for the Characterization of Impedance Spectra. *Batteries* **2019**, *5* (3), 53. https://doi.org/10.3390/batteries5030053.

[5] N. Schluter; J. Schmitt; R. Hanke. Finding the Optimal Regularization Parameter in Distribution of Relaxation Times Analysis. *ChemElectroChem* **2019**, *6*, 6027–6035. https://doi.org/10.1002/celc.201901863.

[6] M. Saccoccio; T. H. Wan; C. Chen; F. Ciucci. Optimal Regularization in Distribution of Relaxation Times Applied to Electrochemical Impedance Spectroscopy: Ridge and Lasso Regression Methods - A Theoretical and Experimental Study. *Electrochim. Acta* **2014**, *147*, 470–482. https://doi.org/10.1016/j.electacta.2014.09.058.

[7] E. Ivers-Tiffee; A. Weber. Evaluation of Electrochemical Impedance Spectra by the Distribution of Relaxation Times. *J. Ceram. Soc. Jpn.* **2017**, *125* (4), 193–201. https://doi.org/10.2109/jcersj2.17015.

[8] B. A. Boukamp. Analysis and Application of Distribution of Relaxation Times in Solid State Ionics. *Solid State Ionics* **2017**, *302*, 12–18. https://doi.org/10.1016/j.ssi.2017.02.024.

[9] P. Cordoba-Torres. A General Theory for the Impedance Response of Dielectric Films with a Distribution of Relaxation Times. *Electrochim. Acta* **2018**, *282*, 892–904. https://doi.org/10.1016/j.electacta.2018.06.161.

[10] J. Macutkevic; J. Banys; A. Matulis. Determination of the Distribution of the Relaxation Times from Dielectric Spectra. *Nonlinear Anal. Model. Control* **2004**, *9* (1), 75–88.

[11] G. H. Golub; M. Heath; G. Wahba. Generalized Cross-Validation as a Method for Choosing a Good Ridge Parameter. *Technometrics* **1979**, *21* (2), 215–223. https://doi.org/10.1080/00401706.1979.10489751.

[12] P. C. Hansen; D. P. O'Leary. The Use of the L-Curve in the Regularization of Discrete Ill-Posed Problems. *SIAM J. Sci. Comput.* **1993**, *14* (6), 1487–1503. https://doi.org/10.1137/0914086.

[13] E. Tuncer; S. M. Gubanski. On Dielectric Data Analysis: Using the Monte Carlo Method to Obtain Relaxation Time Distribution and Comparing Non-Linear Spectral Function Fit. *IEEE Trans. Dielectr. Electr. Insul.* **2001**, *8* (2), 310–318. https://doi.org/10.1109/94.922261.

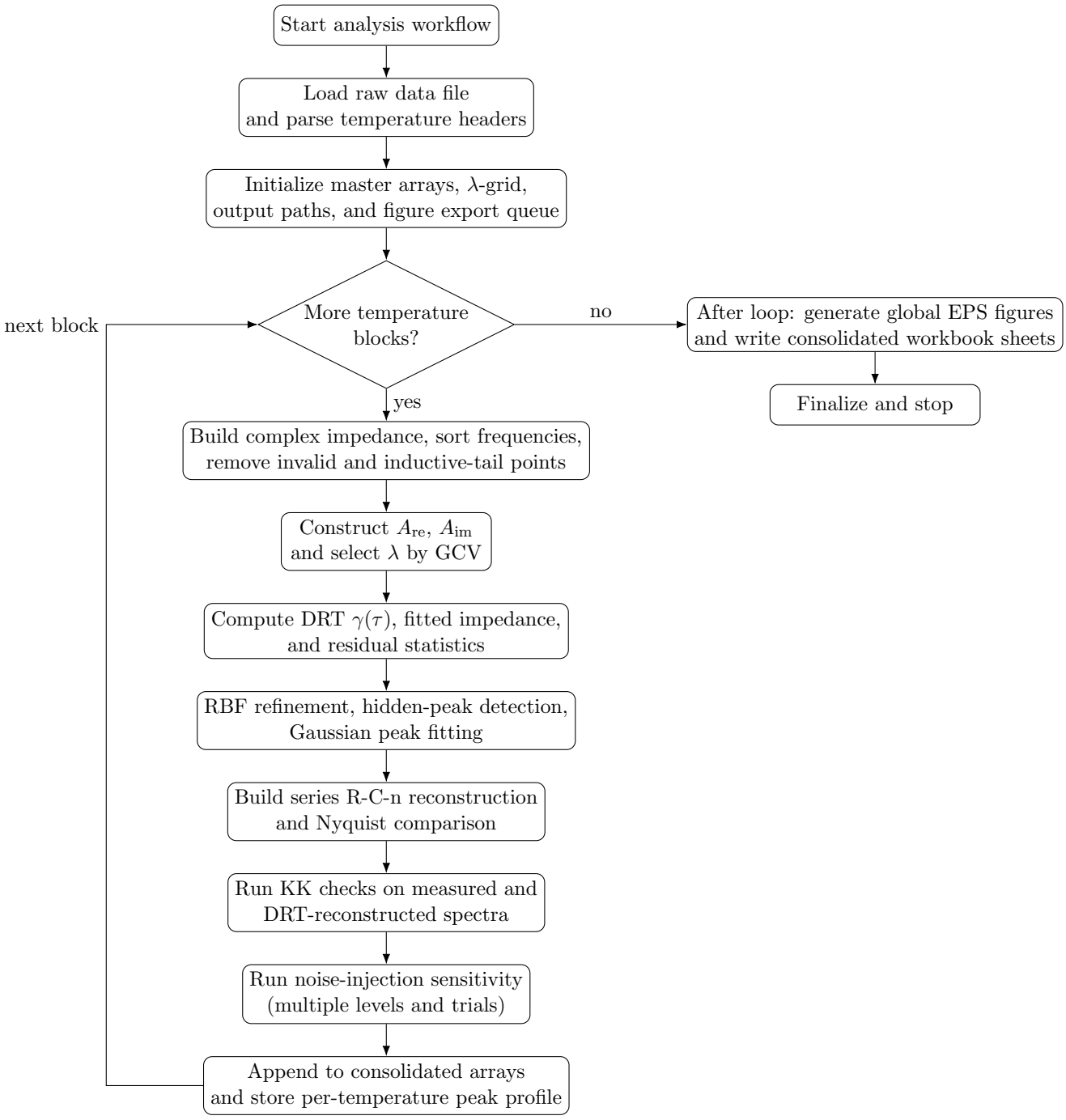

Figure 1: Generalized end-to-end analysis flow with explicit per-temperature loop, branch conditions, and post-loop export stage.

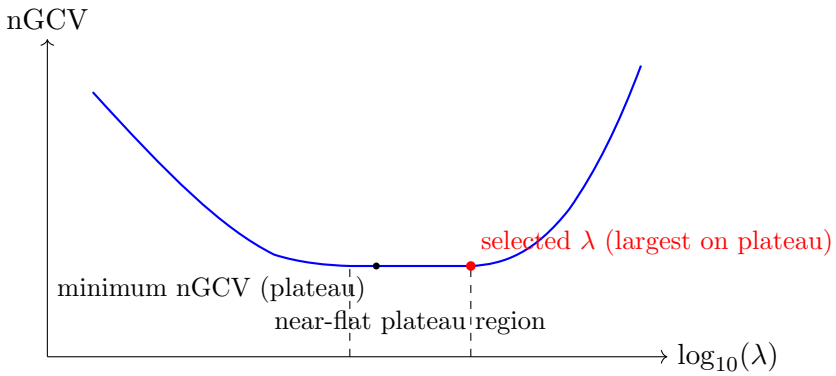

Figure 2: Illustrative nGCV-vs-λ profile showing the plateau-based selection policy used in the MATLAB workflow.

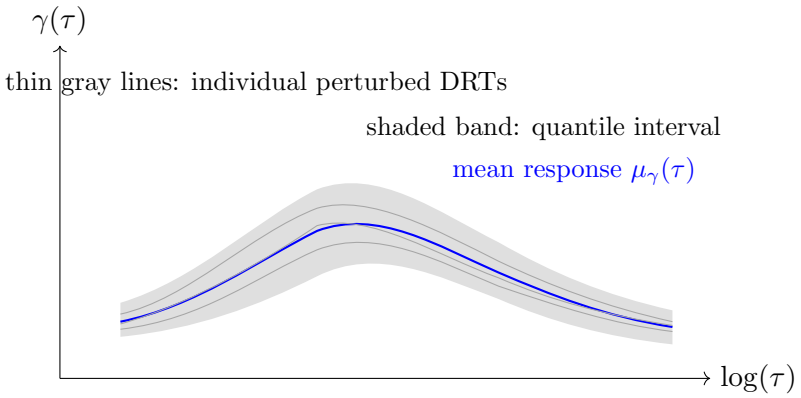

Figure 3: Illustrative uncertainty view. In the current script, robustness is reported through noise-injection residual statistics; this schematic still shows the general idea that stable inversions should vary smoothly under perturbations.

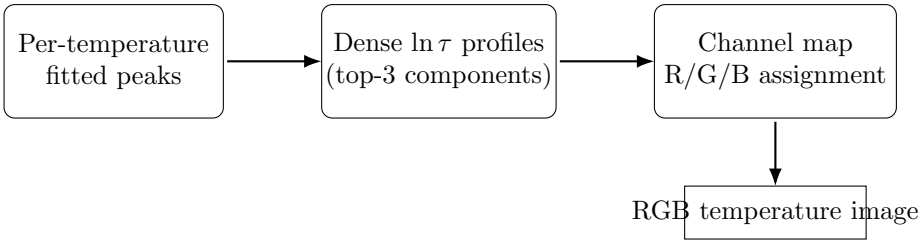

Figure 4: Image-construction pipeline used for the RGB peak map in the MATLAB workflow.

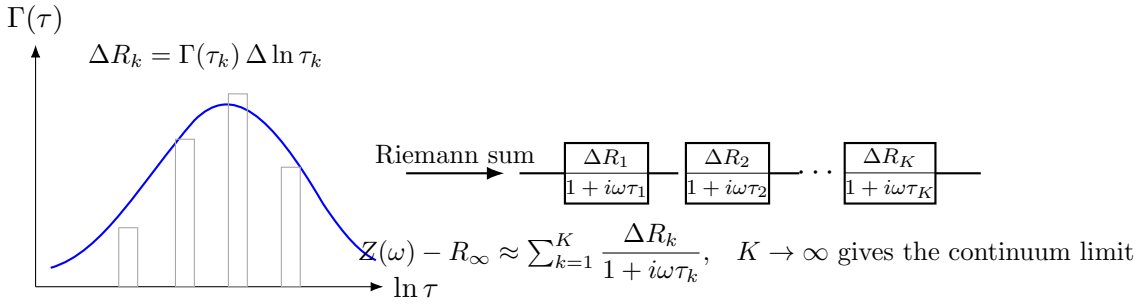

Figure 5: ZARC as a continuum sum of ZAP elements: binning the DRT density over $\ln \tau$ yields a finite Debye/ZAP series, whose refinement limit gives the exact integral representation.

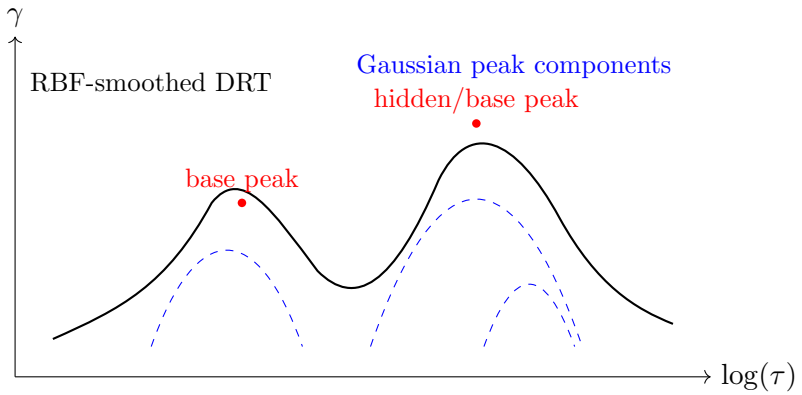

Figure 6: Illustrative peak-refinement stage: RBF-smoothed DRT profile and Gaussian component decomposition used to export per-peak parameters.